

- [15] M. Richardson and P. Domingos. Markov logic networks. *Machine learning*, 62(1-2):107–136, 2006.
- [16] S. Richardson, P. Domingos, and M. S. H. Poon. The alchemy system for statistical relational ai: User manual, 2007.
- [17] T. Rocktäschel and S. Riedel. End-to-end differentiable proving. In *Advances in Neural Information Processing Systems*, pages 3788–3800, 2017.
- [18] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*, chapter 9, pages 322–325. Prentice Hall Press, USA, 3 edition, 2009.
- [19] L. Serafini and A. d. Garcez. Logic tensor networks: Deep learning and logical reasoning from data and knowledge. *arXiv preprint arXiv:1606.04422*, 2016.
- [20] *Stardog*. Available at <https://www.stardog.com/>.
- [21] G. Sutcliffe. The tptp problem library and associated infrastructure. *Journal of Automated Reasoning*, 43(4):337, 2009.
- [22] G. G. Towell and J. W. Shavlik. Knowledge-based artificial neural networks. *Artificial intelligence*, 70(1-2):119–165, 1994.
- [23] *Virtuoso*. Available at <https://virtuoso.openlinksw.com/>.

Supplementary material for Logical Neural Networks

A First-order logical neural networks

A.1 Overview

This section introduces an extension to the representation of the LNN to handle formulae expressed in the first-order logic (FOL) language, and supplements footnote 1 (page 1) in the main paper. The FOL language enables representing a domain in terms of objects that exist in that domain and relations that hold between the objects in that domain. Its vocabulary includes *constant* symbols, *predicate* symbols, *functional* symbols, where constants refer to objects and predicates and functions refer to relations. In addition to the logical connectives in propositional logic, it introduces two *quantifiers*, the *universal* and *existential* quantifiers. Furthermore, some FOL representations introduce an *equality* symbol, which is treated as either a predicate or a logical connective. This section discusses the treatment of First-order LNN without function and equality symbols and treatment of these will be provided in future work. Furthermore, since equality is not handled, we necessarily make the **unique-names assumption**, insisting that every constant symbol refers to a distinct object in the domain.

In First-order LNN, there exist a neuron for each logical connective as before, a connective neuron, and also for each predicate symbol, an atomic neuron. In addition, each neuron keeps a table whose columns are keyed by unique variables appearing in the represented (sub)formulae, and rows keyed by a set of n -element tuples of groundings, where the tuple size n corresponds to the arity of the neuron. The content of the tables are the bounds of each grounding when substituted for the variables. Similar to Negation, quantifiers are modeled as pass-through nodes with no parameters, with special operations for each quantifier type. The bounds of a universal quantifier node are set to the bounds of the grounding with the lowest upper bound, so that if all of its groundings are True then the universal statement is also True. And the bounds of the existential quantifier are set to the bounds of the grounding with highest lower bound, so that if this lower bound is True the existential statement will be True when at least one of its groundings is True.

Computation of bounds throughout the network proceeds as before, with each grounding treated separately. This method of extending to FOL is similar to approaches that reduce inference in classical first-order logic to propositional logic. Each grounding in each formulae is treated as a proposition.

A.2 Inference in first-order logical neural networks

Section A.1 introduced the representation of First-order logical neural networks, where constants, predicates and quantifiers were introduced. Instead of proposition neurons there are predicate neurons, or atomic neurons, and in addition to connective neurons there are also neurons for quantifiers. In inference for First-order LNN all neurons return tables of groundings and their corresponding bounds. Neural activation functions are then modified to perform joins over columns pertaining to shared variables while computing truth value bounds for the associated grounding. Inverse activation functions are modified similarly, but must also reduce results over any columns pertaining to variables absent from the target input's corresponding subformula so as to aggregate the tightest bounds. In the special case that tables are keyed by consecutive integers, these computations are equivalent to elementwise broadcast operations on sparse tensors, where each tensor dimension pertains to a different variable.

Inverse computation for quantifiers eliminates a given key column(s) corresponding to quantified variable(s) by reducing with min or max as appropriate. However, a proper treatment of inverse computation for the existential quantifier is more complicated as it would introduce Skolem functions. With functions currently not handled, inverse computation for the existential quantifier only broadcasts its known upper bounds to all key values associated with its column (i.e. variable) and broadcasts

its known lower bounds to a group of new key values identified by each combination of key values associated with any of its columns and vice versa for universal quantifiers.

A.3 Grounding management

Neurons in First-order LNN each have a defined set of variable(s) according to their arity, specifying the number of constants in a grounding tuple. The arity of predicate neurons is usually set beforehand (e.g., informed by a knowledge base ingested by the LNN), and can typically include a variety of nullary (propositional), unary, binary and higher-arity predicates. Connective neurons and quantifiers collect variables from their set of input (or operands) in order of appearance during initialization, where these operands can include atomic neurons, other connective neurons and quantifiers. Variables are collected only once from operands that define repeat occurrences of a specific variable in more than one variable position, unless otherwise specified. Logical formulae can also be defined with arbitrary variable placement across its constituent nodes. A variable mapping operation transforms groundings for enabling truth value lookup in neighboring nodes.

Usually, some formulae initially contain only variables (e.g., axioms asserted corresponding to general rules), leading to neurons with no groundings initially. These neurons must obtain their groundings from their ground operands at initialization or during inference. During inference, each ground neuron propagates its groundings to other neurons with shared variables participating in the same parent neuron, and all operands collectively propagate their groundings to the parent neuron. Similarly, a parent neuron propagates groundings acquired elsewhere to its operands. This grounding management process ensures that constants are propagated throughout the LNN graph in order to compute bounds for relevant queries. Note, however, that this naive grounding management process will propagate all constants, including those not relevant for the query, unnecessarily increasing computation. More efficient methods are discussed in Section A.5 below.

Quantifiers can also have variables and groundings if partial quantification is required for only a subset of variables from the underlying operand, although existential quantification is typically performed on a single variable to produce a propositional truth value associated with the quantifier output. For partial quantification the maximum lower bound of groundings from the quantified variable subset is chosen for existential quantification and assigned to a unique grounding consisting of the remainder of the variables, whereas the minimum upper bound is used for universal quantification. For existential partial quantification True groundings for the quantified variable subset form arguments stored under the grounding of the remaining variable subset, so that satisfying groundings can be recalled.

A.4 Variable binding

Variable binding assigns specific constant(s) to variables of neurons, typically as part of an inference task, such as answering a query. A variable could be bound in only a subset of occurrences within a logical formulae, although the procedure for producing groundings for inference would typically propagate the binding to all occurrences. It is thus necessary to retain the variable even if bound, in order to interact with other occurrences of the variable in the logical formula to perform join operations.

A.5 First-order logical inference

Inference at a connective neuron involves upward and downward pass computations of the associated logical connective for a given set of groundings, whereas inference at a quantifier neuron involves a reduction operation and creation of new groundings in the case of partial quantification. A provided grounding may not be available in all participating operands of an inference operation, where a retrieval attempt would then add the previously unavailable grounding to the operand with Unknown truth value under the open-world assumption. If a proof is offered to a neuron for an unavailable grounding, the proof aggregation would also assume maximally loose starting bounds.

The computational and memory considerations for large knowledge bases with many constants should be taken under consideration, where action may be taken to avoid storing of groundings with unknown bounds. However, inference is a principal means by which groundings are propagated through a logical formula to enable theorem proving, although there are cases where storage can be avoided. In particular, negation can be viewed as a pass-through operation where inference is performed